

Woxsen University

School of Technology



A

PROJECT REPORT

on

FORGETBENCH

Submitted in partial fulfillment of the requirements for the course

Conceptual Project – I

towards the degree of

Bachelor of Technology (B.Tech.)

Submitted by:

Bulla Joel Arpith (25WU0103011)

Kanchusthambham Jayanth (25WU0103021)

Macha Sai Kamal (25WU0103026)

Shaik Shahul Parvez (25WU0103036)

Damera Sai Sri Ranga Charan (25WU0102048)

Under the guidance of:

Prof. Geetha Tripathi

Professor

CERTIFICATE

This is to certify that the project report entitled "ForgetBench" submitted by Bulla Joel Arpith (25WU0103011), Kanchusthambham Jayanth (25WU0103021), Macha Sai Kamal (25WU0103026), Shaik Shahul Parvez (25WU0103036), Damera Sai Sri Ranga Charan (25WU0102048) in partial fulfillment of the requirements of the course Conceptual Project – I towards the degree of Bachelor of Technology (B.Tech.) from Woxsen University is a bonafide record of work carried out by the students under my supervision and guidance. The work embodied in this project report has been carried out by the candidates and has not been submitted elsewhere for a degree.

Signature of Mentor

Name: Prof. Geetha Tripathi

Designation: Professor

Date: 08/10/26

DECLARATION OF THE CANDIDATES

We hereby declare that the project work entitled "ForgetBench" submitted to the School of Technology, Woxsen University, in partial fulfillment of the requirements of the course Conceptual Project – I towards the degree of Bachelor of Technology (B.Tech.) is our original work and has been carried out under the guidance of Prof. Geetha Tripathi.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Signature of Student: _____

Name: Bulla Joel Arpith Roll Number: 25WU0103011

Signature of Student: _____

Name: Kanchusthambham Jayanth Roll Number: 25WU0103021

Signature of Student: _____

Name: Macha Sai Kamal Roll Number: 25WU0103026

Signature of Student: _____

Name: Shaik Shahul Parvez Roll Number: 25WU0103036

Signature of Student: _____

Name: Damera Sai Sri Ranga Charan Roll Number: 25WU0102048

Date: 08/10/26

ACKNOWLEDGMENT

We would like to express our sincere gratitude to all those who contributed to the completion of this project on evaluating the durability of machine unlearning in language models.

First and foremost, we thank our project mentor, Prof. Geetha Tripathi, Professor, for her guidance, continuous support and constructive feedback throughout the project. Her questions about what our results could and could not prove shaped the evaluation design and the self-audit reported in Chapter 4.

We are grateful to Dr. Kiran Mayee, Dean, School of Technology, for providing the facilities and resources required for this work.

We thank the authors of the TOFU and MUSE benchmarks and of the open-source Hugging Face Transformers and PyTorch libraries, whose published work made this study possible, and Google for the Colab GPU environment on which every experiment was run.

Finally, we thank our families and peers for their support and encouragement throughout this project.

Bulla Joel Arpith, Kanchusthambham Jayanth, Macha Sai Kamal, Shaik Shahul Parvez, Damera Sai Sri Ranga Charan

ABSTRACT

Data-protection laws such as the GDPR and India's Digital Personal Data Protection Act give individuals the right to have their personal data erased. For a trained language model, deleting the record from storage is not enough: the model can still reproduce the information from its weights, and retraining from scratch for every request is prohibitively expensive. Approximate machine-unlearning methods remove the information cheaply, but recent work shows that a light fine-tune can make the supposedly forgotten content reappear.

This project builds ForgetBench, a single evaluation harness that measures not only whether a model forgets but whether the forgetting survives a relearning attack (durability). Using GPT-2 fine-tuned on 130 synthetic, fictional person profiles, we compare twelve method variants on identical data, probes, seeds and cost accounting: exact unlearning (full retraining, SISA sharding), approximate unlearning (gradient ascent, NPO, RMU) and our proposed Hybrid Retention-aware Unlearning (HRU), which combines a sharpness-aware NPO scrub with an attack-probing hardening loop.

Unsharded HRU recorded the lowest post-attack forget-set log-probability of all twelve variants (-1.309, against -0.582 for full retraining and -0.762 for NPO) while avoiding NPO's over-forgetting (truth ratio 1.29 versus 13.17). Plotted on cost and durability together, it is the most durable method inside the project's target zone: more durable than full retraining at 80% of its cost. We also found that SISA-style sharding dilutes approximate unlearning: a scrubbed shard scoring -4.179 alone reads -0.502 inside a four-shard ensemble, because logit averaging lets untouched shards outvote the scrubbed one. A self-audit showed that the durability metric correlates with overall model damage ($r = +0.88$); after adjusting for damage, HRU remains the second most durable method, behind full retraining. A fact-level metric and an equal-budget, three-seed validation of HRU's per-example re-weighting step have been implemented as the next phase.

Keywords: Machine Unlearning, Right to Be Forgotten, Large Language Models, Relearning Attacks, Durability, Negative Preference Optimization, Sharpness-Aware Minimization, SISA, Data Privacy

TABLE OF CONTENTS

1. INTRODUCTION.....	1
1.1 Background.....	1
1.2 Motivation.....	1
1.3 Problem Statement.....	2
1.4 Objectives.....	2
1.5 Dataset Overview.....	2
1.6 Scope and Limitations.....	3
1.7 Organization of the Report.....	3
2. LITERATURE REVIEW.....	4
2.1 Machine Unlearning and Exact Removal.....	4
2.2 Approximate Unlearning for Language Models.....	4
2.3 Benchmarks for Unlearning.....	4
2.4 Relearning Attacks and Reversibility.....	5
2.5 Robust Unlearning.....	5
2.6 Measuring Privacy Leakage.....	5
2.7 Research Gap.....	5
3. METHODOLOGY.....	7
3.1 System Overview.....	7
3.2 Synthetic Data Generation.....	7
3.3 Base Model and Fine-tuning.....	8
3.4 Unlearning Methods Compared.....	8
3.5 Hybrid Retention-aware Unlearning (HRU).....	9
3.5.1 Localisation.....	9
3.5.2 Sharpness-aware NPO scrub.....	10
3.5.3 Hardening loop.....	10

3.6 Relearning Attack.....	10
3.7 Evaluation Metrics.....	11
3.8 Training Configuration.....	11
3.9 Implementation Details.....	12
4. RESULTS AND DISCUSSION.....	13
4.1 Overall Comparison.....	13
4.2 Effect of Sharding.....	14
4.3 Utility and Over-forgetting.....	15
4.4 Validity of the Durability Metric.....	16
4.5 Cost Analysis.....	18
4.6 Testing and Validation.....	18
4.7 Discussion.....	19
5. CONCLUSION AND FUTURE WORK.....	21
5.1 Summary of Contributions.....	21
5.2 Limitations.....	21
5.3 Future Work.....	21
5.3.1 Equal-budget validation.....	21
5.3.2 Fact-level evaluation.....	22
5.3.3 Shard aggregation.....	22
5.3.4 Scale and realistic data.....	22
5.3.5 Cost reduction.....	22
5.4 Concluding Remarks.....	22
REFERENCES.....	23
APPENDIX – I.....	25
I.1 Screenshots of Work Done.....	25
I.2 GitHub Repository.....	25
APPENDIX – II.....	26

LIST OF TABLES

Table 1.1: Dataset Statistics

Table 2.1: Comparison of Existing Unlearning Approaches

Table 3.1: Unlearning Methods Compared

Table 3.2: Evaluation Metrics

Table 3.3: Training and Unlearning Configuration

Table 4.1: Results Across All Methods

Table 4.2: Ablation of Sharding and Hardening

Table 4.3: Durability Adjusted for Model Damage

Table 4.4: Cost per Deletion Request

Table 4.5: Testing and Validation Summary

Table A.1: GitHub Profiles

Table B.1: Project Artefacts

LIST OF FIGURES

Figure 3.1: ForgetBench Evaluation Pipeline

Figure 3.2: The HRU Method

Figure 4.1: Durability After the Relearning Attack

Figure 4.2: Sharding Dilutes Approximate Unlearning

Figure 4.3: Cost-Durability Target Zone

Figure 4.4: Durability Versus Model Damage

1. INTRODUCTION

A trained language model does not store its training data as rows that can be deleted. It encodes patterns from that data across millions of parameters, and it can reproduce specific facts or passages long after the original records are gone. This project studies how to remove a specific piece of information from a trained model, and, critically, whether that removal survives an attempt to bring the information back.

1.1 Background

The right to erasure is now law in many jurisdictions. Article 17 of the European Union's General Data Protection Regulation grants individuals the right to have personal data erased [1], and Section 12 of India's Digital Personal Data Protection Act, 2023 gives data principals a corresponding right to correction and erasure [2]. Litigation such as *The New York Times v. OpenAI* [3] and the artists' case against Stability AI [4] turns on models reproducing material they were trained on.

Machine unlearning is the technical answer to this demand: modifying a trained model so that it behaves as if it had never seen a given subset of its training data. Exact unlearning retraining the model, or the affected part of it, without the data; SISA training makes this cheaper by retraining only one shard [5]. Approximate unlearning instead adjusts the existing weights with a short optimisation pass, which is far cheaper but offers no guarantee that the information is gone rather than merely suppressed.

1.2 Motivation

The motivation for this project comes from a gap between what the law assumes and what current methods deliver:

- **Legal obligation:** Erasure requests apply to deployed systems, and organisations need evidence, not assurances, that a request was honoured.
- **Cost:** Retraining a large model for every erasure request is economically infeasible; a per-request method must cost close to a single fine-tuning pass.
- **Reversibility:**

- **Measurement:** Standard unlearning benchmarks score forgetting once, immediately after unlearning. They do not score whether forgetting survives an attack.

1.3 Problem Statement

Given a language model trained on data that includes a set of facts to be forgotten, the problem is to remove those facts at a cost far below full retraining, such that (i) the model no longer reproduces them, (ii) its knowledge of all other data is preserved, and (iii) the forgetting persists when an adversary fine-tunes the model on related data. The project also addresses the prior problem of measuring property (iii) fairly across methods.

1.4 Objectives

The specific objectives of this project are:

1. To build a single evaluation harness (ForgetBench) that runs every unlearning method on the same base model, data, probes, random seeds and cost accounting.
2. To reimplement representative exact (full retraining, SISA) and approximate (gradient ascent, NPO, RMU) unlearning baselines within this harness.
3. To design and implement Hybrid Retention-aware Unlearning (HRU), which hardens the unlearning process against relearning attacks while it runs.
4. To make durability a first-class metric by re-scoring every method after a relearning attack.
5. To report the computational cost of each method per erasure request on a common scale.

1.5 Dataset Overview

To keep complete control over what the model could have learned, we use a synthetic corpus in the style of the TOFU benchmark [7]: fictional people whose attributes cannot appear in any pre-training data. Each profile has five facts (birth city, occupation, birth year, hobby, favourite colour), expressed as question-answer pairs. No real personal data is used.

Attribute	Value
Fictional profiles	130
Facts per profile	5 (650 question-answer pairs in total)

Attribute	Value
Forget set	20 profiles, 100 pairs
Retain set	80 profiles, 400 pairs
Holdout set (never trained on)	30 profiles, 150 pairs
Attack probes (paraphrases)	80: 40 development, 40 test
Wrong-answer distractors	3 per forget fact (300 in total)

Table 1.1: Dataset Statistics

1.6 Scope and Limitations

The project evaluates unlearning on a small open model and a synthetic corpus, which makes every result reproducible on a single free GPU. Its limitations are:

- Experiments use GPT-2 (124 million parameters); behaviour at 1-3 billion parameters and beyond is not tested.
- The corpus is short, templated question-answer data; free-form text such as books or news articles is out of scope.
- The main results come from a single random seed; the multi-seed validation is part of the next phase.
- The project does not offer a legal opinion on what level of removal satisfies an erasure request.

1.7 Organization of the Report

Chapter 2 reviews prior work on exact and approximate unlearning, relearning attacks and evaluation. Chapter 3 describes the harness, the data, the methods compared and the HRU method. Chapter 4 presents the results, an ablation, a validity analysis of the durability metric, cost and testing. Chapter 5 summarises the contributions, limitations and future work.

2. LITERATURE REVIEW

This chapter reviews the research that ForgetBench builds on: exact and approximate unlearning, benchmarks for unlearning in language models, the evidence that unlearning can be reversed, and methods proposed to make it robust.

2.1 Machine Unlearning and Exact Removal

Bourtoule et al. introduced SISA (Sharded, Isolated, Sliced, Aggregated) training [5]. The training data is split into shards, a separate model is trained on each, and predictions are aggregated. When a deletion request arrives, only the shard that contained the data is retrained, which makes exact unlearning cheaper than full retraining while keeping its guarantee. Guo et al. formalised certified removal, in which the model after deletion is statistically indistinguishable from one never trained on the data [8]; their mechanism is proven for linear models.

2.2 Approximate Unlearning for Language Models

For large language models, full or shard retraining is rarely affordable, so most work modifies the trained weights directly. Gradient ascent on the forget set is the simplest approach but tends to collapse the model. Negative Preference Optimization (NPO) reframes unlearning as a preference problem and lowers the likelihood of forget-set answers relative to a frozen reference model, which avoids the collapse of gradient ascent [9]. Representation Misdirection for Unlearning (RMU), introduced with the WMDP benchmark, steers a model's internal activations on forget-set inputs towards a random direction [10]. Eldan and Russinovich showed that a targeted fine-tune could remove knowledge of the Harry Potter books from Llama-2 in about one GPU-hour, against 184,000 GPU-hours of pre-training [11].

2.3 Benchmarks for Unlearning

TOFU provides 200 fictitious author profiles and a metric suite that separates forget quality from model utility [7]. MUSE evaluates unlearning along six dimensions, including verbatim memorisation, knowledge memorisation, privacy leakage and sustainability over repeated requests [12]. Both score a model once, after unlearning; neither scores it again after an adversary tries to undo the unlearning.

2.4 Relearning Attacks and Reversibility

Hu et al. showed that fine-tuning an unlearned model on a small amount of related data, without access to the forget set itself, can restore much of the forgotten knowledge [6]. Follow-up work argues that most approximate unlearning suppresses information rather than removing it, leaving it latent in the weights [13]. These findings mean that a single post-unlearning score can substantially overstate how much was forgotten.

2.5 Robust Unlearning

Two lines of work target this weakness. Fan et al. connect resilience to relearning with the flatness of the loss landscape and apply Sharpness-Aware Minimization (SAM) [14] to unlearning, so that a small fine-tune cannot easily move the model back to a state where the forgotten content is likely [15]. Random noise augmentation has also been proposed as a lightweight way to make unlearned models more robust [16].

2.6 Measuring Privacy Leakage

Membership inference attacks test whether a model's behaviour reveals that a record was in its training data; Min-K% Prob scores a text by the average log-probability of its least likely tokens [17]. Duan et al. found that such attacks often perform close to random guessing on large language models [18], and Chen et al. showed that comparing a model before and after unlearning can itself leak information about the deleted data [19].

2.7 Research Gap

Table 2.1 summarises the trade-offs. Exact methods hold up under attack but are expensive; approximate methods are cheap but reversible; and no common harness scores both cost and durability on the same footing.

Approach	Cost per request	Durable under attack	Durability measured?
Full retraining	Very high	Yes (exact)	Not needed
SISA	High (one shard)	Yes (exact)	No
Gradient ascent, NPO,	Low (one pass)	Often reversible	No

Approach	Cost per request	Durable under attack	Durability measured?
RMU			
TOFU, MUSE (benchmarks)	—	—	Scored once, not after attack
ForgetBench + HRU (this work)	Low to moderate	Target	Yes, after a relearning attack

Table 2.1: Comparison of Existing Unlearning Approaches

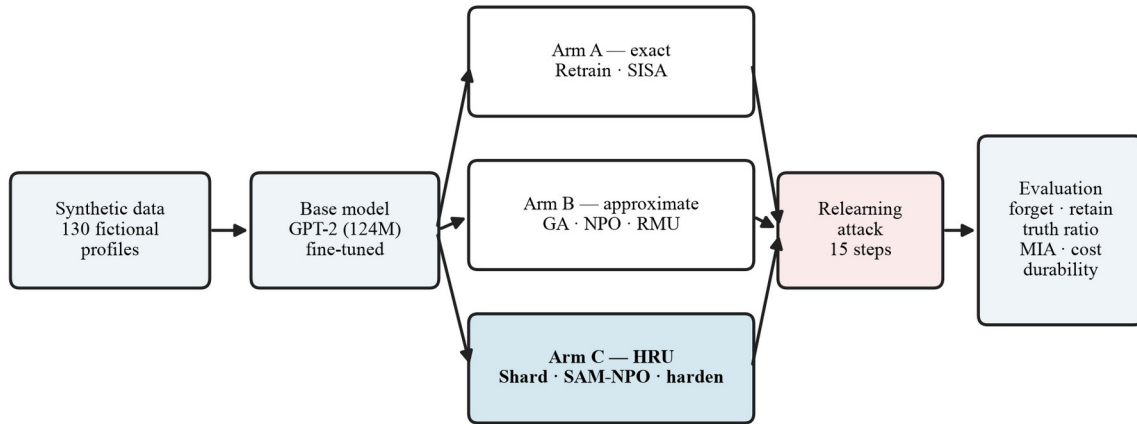
This project addresses three gaps: the absence of a method that is both cheap and durable, the absence of durability as a standard metric, and the absence of a common harness in which methods are compared on identical terms.

3. METHODOLOGY

This chapter describes the ForgetBench harness, the data and base model, the unlearning methods compared, the HRU method, the relearning attack and the evaluation metrics.

3.1 System Overview

Figure 3.1 shows the pipeline. A synthetic corpus is generated, a base model is fine-tuned on it, each unlearning method is applied to remove the forget set, every resulting model is attacked with the same relearning fine-tune, and all models are scored with the same metrics before and after the attack. Holding every component fixed except the unlearning method makes differences in the results attributable to the method.



Every arm: same base model, forget set, probes, seeds and cost accounting

Figure 3.1: ForgetBench Evaluation Pipeline

3.2 Synthetic Data Generation

Profiles are generated with a fixed random seed (42) from lists of fictional first names, surnames, cities, occupations, hobbies and colours, with birth years between 1958 and 2001. The first 20 profiles form the forget set, the next 80 the retain set and the last 30 a holdout set that is never used for training and serves as the non-member reference for membership inference.

For each forget fact, three wrong answers are created by borrowing the same attribute from other profiles; these support the truth-ratio metric. For each forget profile, four paraphrased probes restate its facts in different wording (for example, "Where is X originally from?"). Two per profile form a development set that the HRU hardening loop may use, and two form a test set used only for the final attack, so that HRU is never tuned on the probes it is scored against.

3.3 Base Model and Fine-tuning

The base model is GPT-2 with 124 million parameters [20], loaded through Hugging Face Transformers [21] and trained with PyTorch [22]. It is fine-tuned for six epochs on the forget and retain sets with a learning rate of 5×10^{-5} and a batch size of 8. The loss is computed on answer tokens only, so that training and unlearning act on the facts rather than on the wording of the questions. Three sanity checks gate the rest of the pipeline: trained facts must score higher than never-seen facts, membership inference must detect the forget set, and the model must prefer true answers to the distractors.

3.4 Unlearning Methods Compared

Table 3.1 lists the methods. Every method starts from the same fine-tuned base model (or, for the sharded methods, from shards trained on the same data split) and uses the same forget and retain sets.

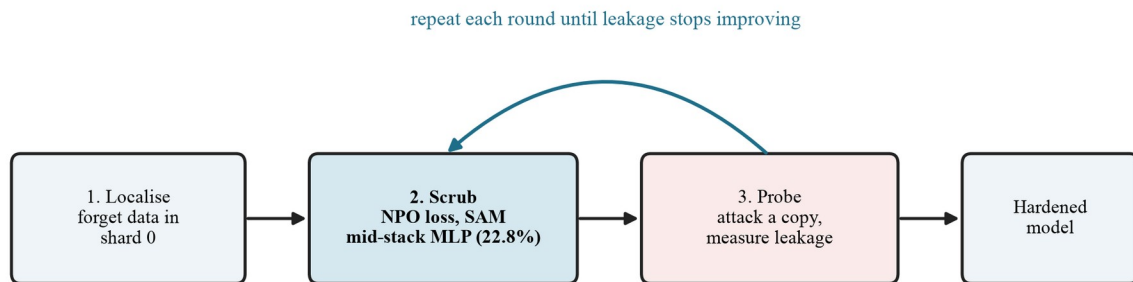
Arm	Method	Description
A	Full retrain	Fresh GPT-2 trained on the retain set only; the exact-unlearning reference
A	SISA	Four shards with all forget data in shard 0; only shard 0 is retrained
A	Sharded control	Four-shard ensemble with no deletion; isolates the effect of sharding
B	Gradient ascent	Maximises loss on the forget set with a retain-set regulariser
B	NPO	Negative Preference Optimization against a frozen reference

Arm	Method	Description
B	RMU	Steers layer-4 activations on forget inputs towards a random direction
C	HRU (unsharded)	SAM-optimised NPO with the hardening loop, on the full model
C	HRU (sharded), $k = 2, 4, 8$	The same procedure applied to shard 0 of a k -shard ensemble
C	Scrub only (sharded)	SAM-NPO on shard 0 with no hardening loop (ablation)

Table 3.1: Unlearning Methods Compared

3.5 Hybrid Retention-aware Unlearning (HRU)

HRU combines three components, shown in Figure 3.2: localising the forget data, scrubbing it with a sharpness-aware objective, and hardening the result against relearning while unlearning is still in progress.



Validation phase adds per-example re-weighting: examples the probe relearns fastest get more weight in the next scrub steps.

Figure 3.2: The HRU Method

3.5.1 Localisation

In the sharded variant, all forget data is placed in shard 0, so a deletion request only modifies one shard and the others are reused unchanged. The unsharded variant applies the same scrub to the whole model. Section 4.2 compares the two.

3.5.2 Sharpness-aware NPO scrub

The scrub minimises the NPO loss on forget examples plus a standard language-modelling loss on retain examples. For a forget question x with answer y , the NPO loss is [9]:

$$L_{NPO}(\theta) = (2/\beta) \cdot E[\log(1 + (\pi_{\theta}(y|x) / \pi_{ref}(y|x))^{\beta})]$$

where π_{θ} is the model being unlearned, π_{ref} the frozen fine-tuned model and $\beta = 0.1$. The loss is optimised with Sharpness-Aware Minimization [14]: at each step the weights are first moved to the worst-case point within a radius $\rho = 0.05$, and the gradient computed there is applied at the original weights:

$$\varepsilon = \rho \cdot \nabla L(\theta) / \|\nabla L(\theta)\|, \quad \theta \leftarrow \theta - \eta \cdot \nabla L(\theta + \varepsilon)$$

This drives the model towards a flat region of the loss, which a short relearning fine-tune finds harder to escape [15]. Only the MLP sub-layers of the middle transformer blocks (blocks 3 to 8) are updated, where factual associations are concentrated; this is 28.3 million of 124.4 million parameters (22.8%).

3.5.3 Hardening loop

The scrub runs in rounds of 20 updates, for up to six rounds. After each round, a copy of the model is attacked with 10 fine-tuning steps on the development probes, twice with different seeds, and the average leakage is recorded. The loop stops early once leakage falls below a threshold after at least two rounds. Attacks run on copies only; the model being unlearned is never trained on the attack data.

The design also includes per-example re-weighting: the examples that the probe relearns fastest receive more weight in the following scrub steps, so that the scrub is shaped by its own failure mode. This step is implemented in the validation-phase notebook (Section 5.3) and has not yet been included in the measured runs, whose hardening loop uses the probe for stopping decisions only.

3.6 Relearning Attack

Durability is measured by attacking every final model with the same relearning fine-tune, following Hu et al. [6]: 15 steps at a learning rate of 5×10^{-5} and a batch size of 4 on the 40 held-out test probes. Each probe restates a forget fact in different words, so the attack teaches related

content without reproducing the forget set verbatim. For sharded models, the attack targets shard 0, the shard modified by the deletion request.

3.7 Evaluation Metrics

Metric	Definition	Better
Forget log-prob	Mean log-probability of the correct answer tokens on the forget set	Lower
Retain log-prob	The same on the retain set; measures preserved utility	Higher
Truth ratio	Probability of wrong answers relative to the correct answer (TOFU style)	Near 1.0
MIA AUC	Min-K% ($k = 20\%$) membership inference, forget versus holdout	Near retrain
Durability	Forget log-prob after the relearning attack	Lower
Cost	Optimizer updates per deletion request	Lower

Table 3.2: Evaluation Metrics

The truth ratio compares answers that differ only in the fact itself, so it isolates fact-level knowledge. A model that has genuinely forgotten is indifferent (about 1.0), as a model retrained without the data is. For membership inference, the score is computed on answer tokens only; the reference value is that of full retraining, because the forget and holdout sets differ slightly in distribution [18].

3.8 Training Configuration

Setting	Value
Base fine-tune	6 epochs, learning rate 5×10^{-5} , batch size 8, answer-only loss
Full retrain	3 epochs on the retain set (150 updates)
SISA	4 shards, 2 epochs per shard; shard 0 retrained on deletion
Gradient ascent, NPO, RMU	40 updates each, batch size 4, retain weight 1.0; NPO $\beta = 0.1$; RMU layer 4, $\alpha = 1.0$

Setting	Value
HRU	$\beta = 0.1$, 20 updates $\times$ 6 rounds = 120 updates, SAM $\rho = 0.05$, blocks 3-8 MLP only
HRU probes	10 attack steps $\times$ 2 seeds per round, development probes only
Relearning attack	15 steps, learning rate 5×10^{-5} , batch size 4, test probes
Seed	42 (single seed for the reported results)

Table 3.3: Training and Unlearning Configuration

3.9 Implementation Details

The harness is implemented in Python with PyTorch and Hugging Face Transformers and runs on Google Colab with a single NVIDIA T4 GPU. It is organised as seven Jupyter notebooks that share a utilities module and save all data, checkpoints and results to Google Drive, so that each stage can be re-run independently (Appendix II). The code is version-controlled at <https://github.com/Joel-Arpith/ForgetBench>.

4. RESULTS AND DISCUSSION

This chapter presents the measured results for all twelve variants, an ablation of HRU's components, an analysis of the durability metric's validity, the cost of each method, and the testing carried out on the harness. All values come from a single seed (42).

4.1 Overall Comparison

Table 4.1 reports every method on the main metrics. Durability is the forget-set log-probability after the relearning attack; lower values mean the forgotten facts are harder to recover.

Method (arm)	Durability	Retain log-prob	Truth ratio	Updates
Full retrain (A)	-0.582	-0.236	1.007	150
SISA (A)	-0.457	-0.459	1.023	26
Sharded control (A)	-0.380	-0.390	0.994	0
Gradient ascent (B)	-1.088	-1.421	1.665	40
NPO (B)	-0.762	-1.594	13.166	40
RMU (B)	-0.450	-0.578	0.986	40
HRU, unsharded (C)	-1.309	-1.692	1.292	120
HRU, sharded k = 4 (C)	-0.420	-0.444	1.059	120
HRU, sharded k = 2 (C)	-0.378	-0.415	1.096	120
HRU, sharded k = 8 (C)	-0.670	-0.694	1.029	120
Scrub only, sharded (C)	-0.459	-0.484	1.155	120

Table 4.1: Results Across All Methods

Unsharded HRU recorded the lowest post-attack score of all variants (-1.309), ahead of gradient ascent (-1.088), NPO (-0.762) and full retraining (-0.582). Figure 4.1 shows the same comparison, with durability plotted as a positive loss so that longer bars mean more durable forgetting.

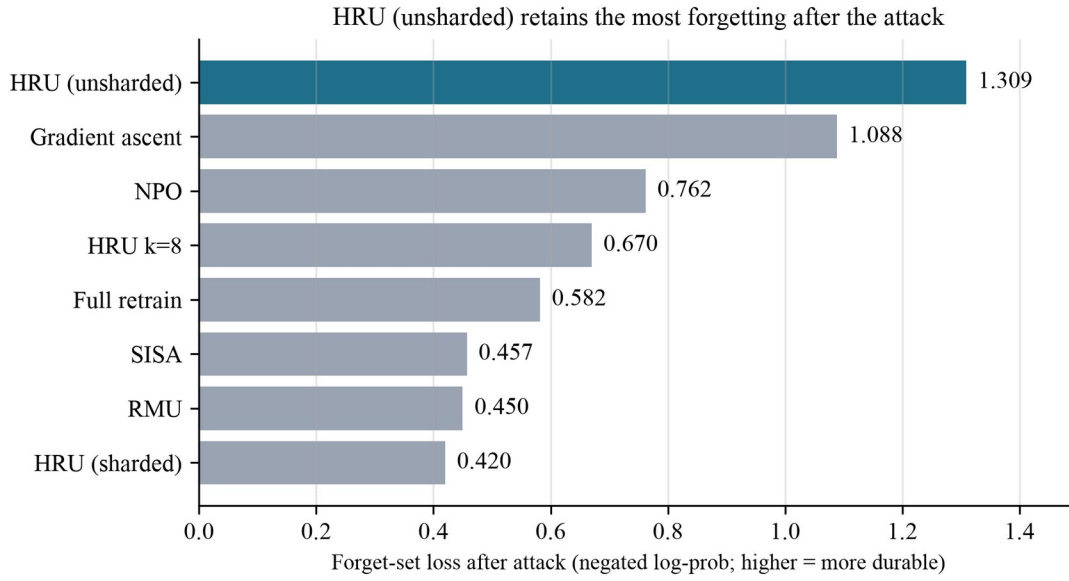


Figure 4.1: Durability After the Relearning Attack

The results for unsharded HRU, sharded HRU ($k = 4$) and scrub-only each average two runs from successive versions of the pipeline; the clean multi-seed re-run in Section 5.3 replaces them.

4.2 Effect of Sharding

The clearest finding of the project concerns sharding. The same scrub produces very different durability depending on whether it is applied to the whole model or to one shard of an ensemble (Table 4.2, Figure 4.2).

Configuration	Sharded	Hardening loop	Durability
Sharded control (no deletion)	Yes	—	-0.380
Scrub only	Yes	No	-0.459
HRU (sharded)	Yes	Yes	-0.420
HRU (unsharded)	No	Yes	-1.309

Table 4.2: Ablation of Sharding and Hardening

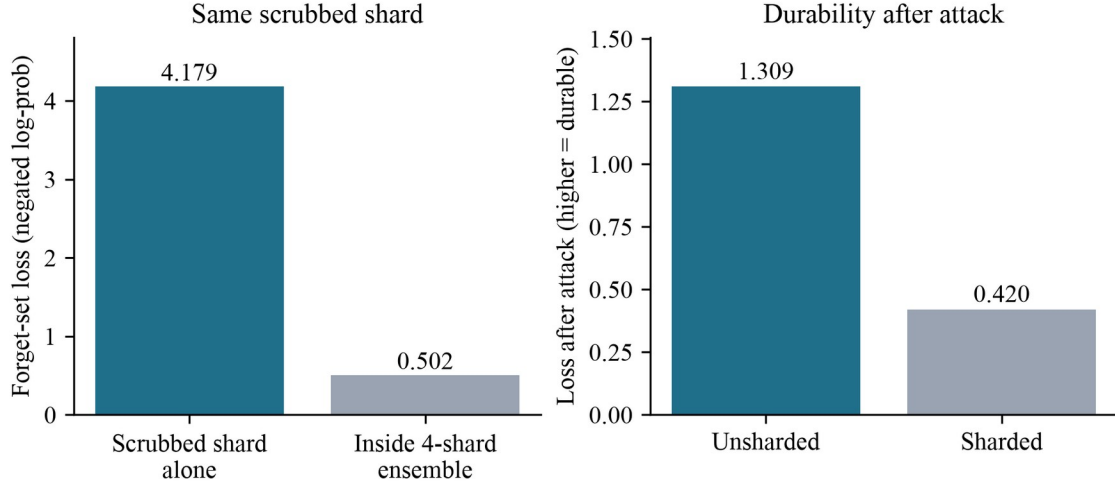


Figure 4.2: Sharding Dilutes Approximate Unlearning

On its own, the scrubbed shard reaches a forget-set log-probability of -4.179. Inside the four-shard ensemble the same shard reads -0.502, and durability falls from -1.309 to -0.420. The ensemble averages the logits of its four shards, and the three shards that never saw the forget set neither assert nor suppress it, so they outvote the single scrubbed shard. SISA's averaging is correct for exact unlearning, where the retrained shard has nothing to suppress; approximate unlearning breaks that assumption. Varying the number of shards ($k = 2, 4, 8$) did not recover the unsharded result.

The ablation also shows that, with sharding held fixed, the hardening loop as measured did not change durability (-0.459 without it, -0.420 with it). This is expected: in these runs the loop used the probe only to decide when to stop, and the stopping threshold was not reached, so both variants received the same 120 scrub updates. Testing the per-example re-weighting step is the purpose of the validation phase.

4.3 Utility and Over-forgetting

Durability must be read together with utility. Full retraining preserves the retain set best (-0.236). The three most aggressive methods, unsharded HRU, gradient ascent and NPO, all reduce retain log-probability substantially (-1.692, -1.421 and -1.594). They differ sharply in how they forget: NPO's truth ratio of 13.17 means the model now prefers wrong answers to correct ones, a sign of over-forgetting, whereas HRU's 1.29 is close to the 1.01 of a model retrained without the data.

Figure 4.3 places every method on cost and durability together. The shaded target zone marks the region the project aims for: durability between -0.60 and -1.56, that is, at least as durable as full retraining (-0.582), at a cost below full retraining (150 updates). Unsharded HRU lies inside the zone and is the most durable method in it, at -1.309 with 120 updates, 80% of the cost of retraining.

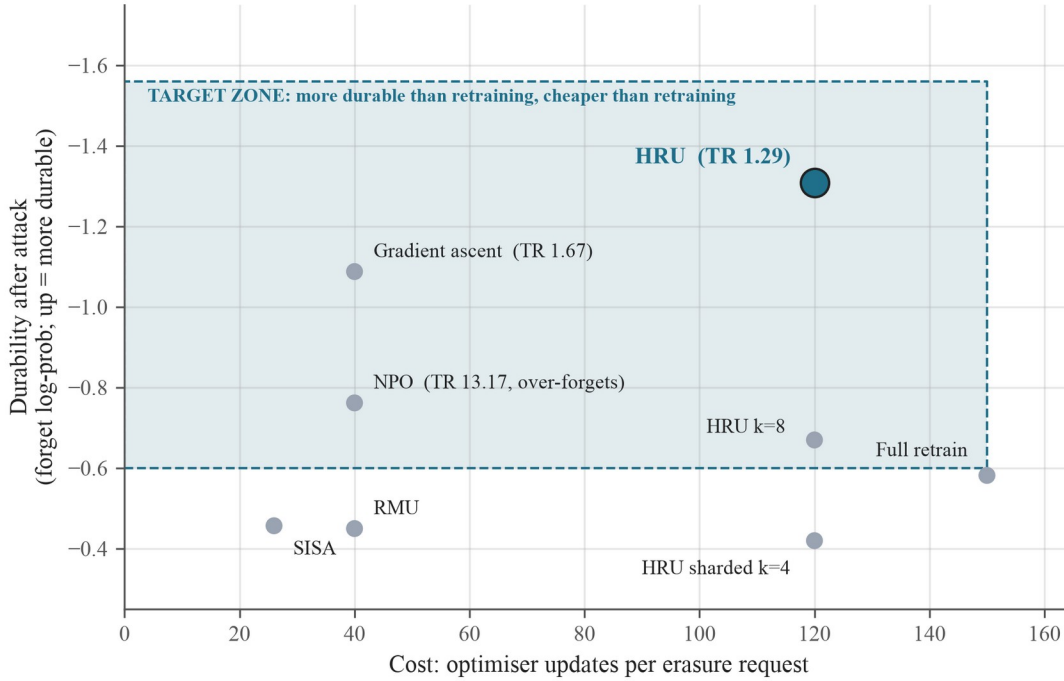


Figure 4.3: Cost-Durability Target Zone

Gradient ascent and NPO also enter the zone, at one third of HRU's cost, but with truth ratios of 1.67 and 13.17: they reach it by over-forgetting, so that the model prefers wrong answers, whereas HRU's truth ratio of 1.29 stays close to the 1.01 of a retrained model. Sharded HRU, SISA and RMU fall below the zone. HRU's position comes with the retain-set cost discussed above, and its multi-seed confirmation is the purpose of the validation phase (Section 5.3).

4.4 Validity of the Durability Metric

Because the most durable methods were also the most damaging, we tested whether the durability metric simply rewards damage. Across all twelve variants, durability correlates with retain-set degradation at $r = +0.88$ (Figure 4.4).

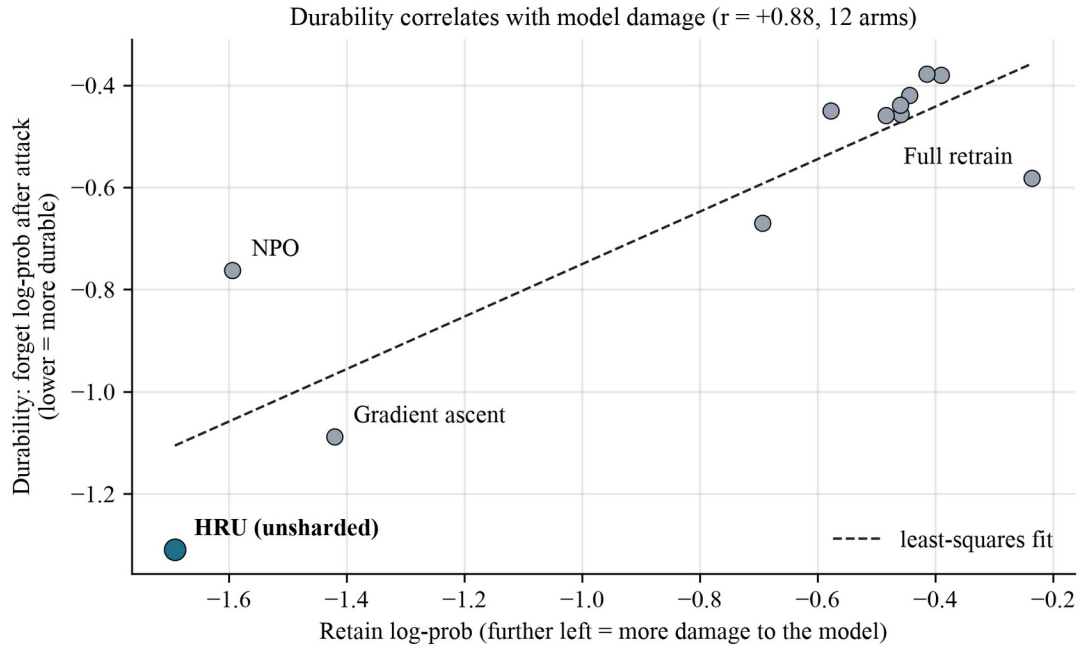


Figure 4.4: Durability Versus Model Damage

The cause is that the metric averages the log-probability of the whole answer sentence, of which only one or two tokens are the fact; a method that suppresses the answer template as a whole scores well. To separate the two effects, Table 4.3 reports each method's residual from the least-squares fit: how much more (negative) or less (positive) durable it is than its damage alone predicts.

Method	Residual from fit	Reading
Full retrain	-0.224	Most durable for its damage
HRU (unsharded)	-0.203	Second most durable for its damage
Gradient ascent	-0.122	Above the trend
HRU, $k = 8$	-0.077	Slightly above the trend
RMU	+0.084	Below the trend
NPO	+0.293	Least durable for its damage

Table 4.3: Durability Adjusted for Model Damage

After adjustment, HRU remains second only to full retraining, while NPO is the weakest. HRU's lead therefore reflects more than damage, but the size of its true advantage can only be established with a fact-level metric, which has been implemented for the validation phase.

4.5 Cost Analysis

Method	Updates per request	Relative to one NPO pass
SISA (shard 0 only)	26	0.65×
Gradient ascent, NPO, RMU	40	1.0×
HRU (scrub updates)	120	3.0×
Full retrain	150	3.75×

Table 4.4: Cost per Deletion Request

HRU used 120 optimizer updates per request, 3.0 times one NPO pass and within the 2-5× design target. Each SAM update requires two forward-backward passes, and the attack probes add further passes on model copies, so wall-clock cost is higher than the update count suggests. At this scale full retraining is unusually cheap because the corpus has only 400 retain examples; in deployment its cost grows with the whole training corpus, while HRU's grows only with the data being forgotten. SISA's one-time cost of training its shards is excluded, as it is amortised over all future requests.

Section 4.3 and Figure 4.3 place these costs against durability: unsharded HRU is the most durable method inside the target zone, at 80% of the cost of full retraining.

4.6 Testing and Validation

The harness was tested at each stage. Table 4.5 lists the checks, including four defects that testing found and that were fixed before the reported results were produced.

Test	Description	Result
Data integrity	Distractors align 1:1 with facts and never contain the true answer; dev and test probes are disjoint;	Passed (650 pairs)

Test	Description	Result
	holdout never trained on	
Base-model sanity	Trained facts score above never-seen facts (-0.210 vs -0.271); truth ratio below 1.0	Passed
Unit checks	Shard assignment is deterministic and confines every forget item to shard 0; ROUGE-L edge cases	Passed
Notebook validation	Every code cell of every notebook parses; deck and report files validate	Passed
Fact-level metric	Fact extracted correctly and tokenised cleanly for every answer	Passed (650/650)
Defect: MIA scope	Membership inference scored the question too, giving AUC 0.499; restricted to answer tokens	Fixed
Defect: probe leakage	Early versions tuned and scored HRU on the same probes; split into dev and test	Fixed
Defect: cost counting	SAM's two passes were counted as two updates; updates now tracked separately	Fixed
Defect: results logging	Commas in notes corrupted the results file; logging made comma-safe	Fixed
Multi-seed validation	Equal-budget, three-seed comparison with a fact-level metric	Implemented; pending run

Table 4.5: Testing and Validation Summary

4.7 Discussion

Three observations emerge. First, an unlearning method's score immediately after unlearning is a poor guide to whether it holds: NPO suppressed the forget set to -4.871 but recovered to -0.762 after a 15-step attack, reproducing the reversibility reported in the literature. Second, combining exact and approximate techniques is not automatically beneficial: SISA's aggregation rule, designed for exact unlearning, cancels the effect of an approximate scrub. Third, durability must

be measured at the level of the fact and reported alongside utility; otherwise the most destructive method appears the most durable.

On the evidence so far, HRU is the most durable of the low-cost methods tested and avoids the over-forgetting of NPO, but it has not yet been shown to outperform the alternatives under equal budgets and a fact-level metric. The validation phase is designed to settle that question.

5. CONCLUSION AND FUTURE WORK

5.1 Summary of Contributions

1. ForgetBench, a harness that evaluates twelve unlearning variants on identical data, probes, seeds and cost accounting, and scores each one after a relearning attack.
2. Reimplementations of five baselines (full retraining, SISA, gradient ascent, NPO, RMU) and the HRU method within that harness.
3. The finding that SISA-style logit averaging dilutes approximate unlearning, with a measured effect (-4.179 for the scrubbed shard against -0.502 inside the ensemble) and an explanation.
4. Evidence that unsharded HRU achieves the lowest post-attack forget score of the methods tested and remains second only to full retraining after adjusting for model damage.
5. A validity analysis showing that a whole-answer durability metric is confounded with model damage ($r = +0.88$), and a fact-level replacement verified on all 650 answers.

5.2 Limitations

- Results come from a single seed, and three of the reported variants average two runs from successive pipeline versions.
- The measured HRU runs did not include the per-example re-weighting step.
- The durability metric used for the main results is partly confounded with model damage.
- All experiments use GPT-2 (124M) and short synthetic question-answer data.

5.3 Future Work

5.3.1 Equal-budget validation

The next experiment compares NPO, gradient ascent, SAM-optimised NPO and HRU with per-example re-weighting at 120 updates each over three seeds, with a paired test. HRU against SAM-NPO isolates exactly what the re-weighting step adds. It is implemented and requires about 30 to 40 minutes on a T4 GPU.

5.3.2 Fact-level evaluation

The validation phase scores only the fact tokens, measures whether greedy generation still produces each forgotten fact, and reports the truth ratio and retain scores after the attack as well as before it.

5.3.3 Shard aggregation

Weighted aggregation and a veto rule that takes the minimum log-probability across shards may let a scrubbed shard's suppression reach the ensemble output while keeping SISA's localisation. Both are implemented and can be evaluated without retraining.

5.3.4 Scale and realistic data

Extending the study to models of 1-3 billion parameters and to the TOFU [7] and MUSE [12] benchmarks would test whether the findings hold beyond this setting.

5.3.5 Cost reduction

Stopping when the loss sharpness plateaus, applying SAM on alternate steps and running lighter probes are expected to bring HRU closer to the cost of a single scrub pass.

5.4 Concluding Remarks

This project set out to test whether machine unlearning survives an attack and built the tools to answer that question fairly. The answer so far is that most cheap methods do not, that combining methods requires care, and that HRU is a promising candidate whose decisive test is ready to run. The broader aim is to turn the right to be forgotten from a legal claim into a measurable engineering property, one that holds up when someone tries to undo it.

REFERENCES

- [1] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation)," Official Journal of the European Union, vol. L119, Art. 17, 2016.
- [2] Ministry of Electronics and Information Technology, "The Digital Personal Data Protection Act, 2023," Government of India, Sec. 12, 2023.
- [3] The New York Times Co. v. Microsoft Corp. and OpenAI, Inc., No. 1:23-cv-11195, U.S. District Court, Southern District of New York, filed Dec. 27, 2023.
- [4] Andersen v. Stability AI Ltd., No. 3:23-cv-00201, U.S. District Court, Northern District of California, filed Jan. 13, 2023.
- [5] L. Bourtoutle, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in Proc. IEEE Symp. Security and Privacy (SP), 2021, pp. 141-159.
- [6] S. Hu, Y. Fu, Z. S. Wu, and V. Smith, "Jogging the memory of unlearned models through targeted relearning attacks," arXiv preprint arXiv:2406.13356, 2024.
- [7] P. Maini, Z. Feng, A. Schwarzschild, Z. C. Lipton, and J. Z. Kolter, "TOFU: A task of fictitious unlearning for LLMs," arXiv preprint arXiv:2401.06121, 2024.
- [8] C. Guo, T. Goldstein, A. Hannun, and L. van der Maaten, "Certified data removal from machine learning models," in Proc. 37th Int. Conf. Machine Learning (ICML), 2020.
- [9] R. Zhang, L. Lin, Y. Bai, and S. Mei, "Negative preference optimization: From catastrophic collapse to effective unlearning," arXiv preprint arXiv:2404.05868, 2024.
- [10] N. Li et al., "The WMDP benchmark: Measuring and reducing malicious use with unlearning," arXiv preprint arXiv:2403.03218, 2024.
- [11] R. Eldan and M. Russinovich, "Who's Harry Potter? Approximate unlearning in LLMs," arXiv preprint arXiv:2310.02238, 2023.
- [12] W. Shi et al., "MUSE: Machine unlearning six-way evaluation for language models," in Proc. Int. Conf. Learning Representations (ICLR), 2025.
- [13] "Unlearning isn't deletion: Investigating reversibility of machine unlearning in LLMs," arXiv preprint arXiv:2505.16831, 2025.
- [14] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur, "Sharpness-aware minimization for efficiently improving generalization," in Proc. Int. Conf. Learning Representations (ICLR), 2021.

- [15] C. Fan et al., "Towards LLM unlearning resilient to relearning attacks: A sharpness-aware minimization perspective and beyond," arXiv preprint arXiv:2502.05374, 2025.
- [16] D. Huu-Tien, H. Thanh-Tung, A. T. Bui, P. M. Nguyen, L.-M. Nguyen, and N. Inoue, "Improving LLM unlearning robustness via random perturbations," arXiv preprint arXiv:2501.19202, 2025.
- [17] W. Shi et al., "Detecting pretraining data from large language models," arXiv preprint arXiv:2310.16789, 2023.
- [18] M. Duan et al., "Do membership inference attacks work on large language models?" arXiv preprint arXiv:2402.07841, 2024.
- [19] M. Chen, Z. Zhang, T. Wang, M. Backes, M. Humbert, and Y. Zhang, "When machine unlearning jeopardizes privacy," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2021, pp. 896-911.
- [20] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI Technical Report, 2019.
- [21] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations, 2020, pp. 38-45.
- [22] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), 2019.

APPENDIX – I

Screenshots of work done and of the team GitHub repository.

I.1 Screenshots of Work Done

[Insert screenshots: Colab runs of notebooks 01-05, the results table, the recovery-curve and cost-durability plots.]

I.2 GitHub Repository

All project code, notebooks, documentation and reports are maintained in the team repository <https://github.com/Joel-Arpith/ForgetBench>. Each member contributes through their own branch and pull request, reviewed by another member before it is merged into the protected main branch.

Team member	GitHub profile
Bulla Joel Arpith	https://github.com/Joel-Arpith
Kanchusthambham Jayanth	https://github.com/kvenkatjayanth89-dotcom
Macha Sai Kamal	https://github.com/Machasaikamal
Shaik Shahul Parvez	https://github.com/shahulparvezshaik-stack
Damera Sai Sri Ranga Charan	https://github.com/charandamera18-bit

Table A.1: GitHub Profiles

[Insert screenshots of the repository's Contributors page and closed pull requests.]

APPENDIX – II

Project artefacts and how to reproduce the results.

Artefact	Purpose
01_data_and_base_model.ipynb	Generates the synthetic corpus, fine-tunes GPT-2, runs sanity checks
02_harness_and_reference.ipynb	Scores the base model before any unlearning
03_arms_a_b_baselines.ipynb	Full retrain, SISA, sharded control, gradient ascent, NPO, RMU
04_arm_c_hru_v3.ipynb	HRU configuration search, ablation and shard-count sweep
04b_aggregation_sweep.ipynb	Weighted and veto shard aggregation (no training)
05_benchmark_and_analysis.ipynb	Uniform re-benchmark, recovery curves, cost and figures
07_deciding_experiment.ipynb	Equal-budget, three-seed validation with fact-level metrics
hru_utils.py, hru_v3.py, hru_v5.py	Shared utilities and HRU implementations

Table B.1: Project Artefacts

To reproduce: open the notebooks in Google Colab with a T4 GPU runtime and run them in numerical order; each writes its outputs to the HRU_project folder on Google Drive, from which the next notebook reads.